# Project Report: Building a Secure Password Generator

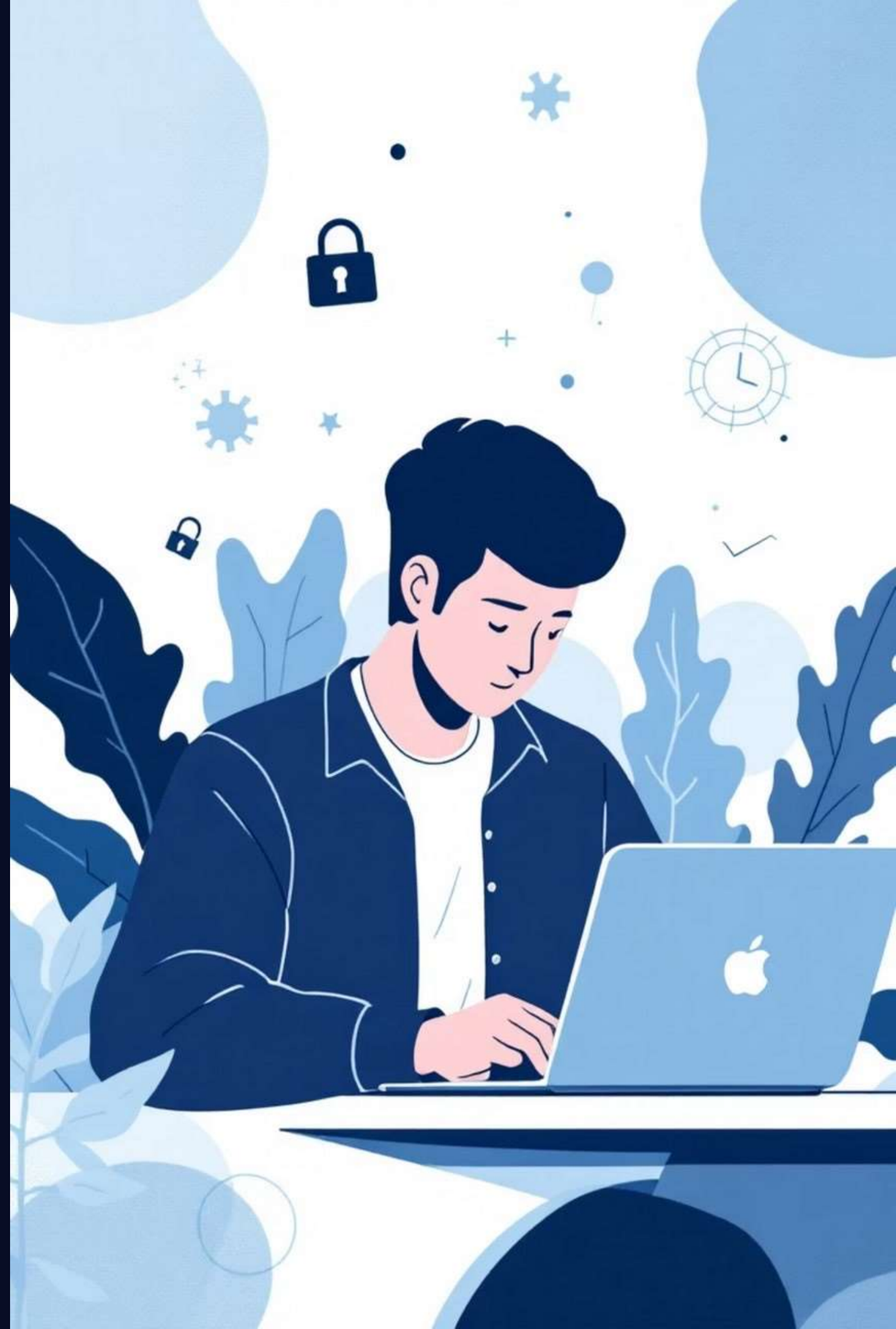
A Project Report for Software Developers and Computer Science Students

Submitted By: Bhavishya Saini

Registration No: 25BEC10036

# Introduction to the Project

This project outlines the development of a Graphical User Interface (GUI) password generator using Python's Tkinter library. Our goal is to create a user-friendly tool that generates strong, random passwords and securely stores them locally.



# Problem Statement: The Need for strong Passwords

### Vulnerable Accounts

Many users rely on weak or recycled passwords, making their online accounts susceptible to breaches.

### Inconsistent Generation

Manually creating complex passwords is time-consuming and often leads to predictable patterns.

### Security Risks

Compromised passwords pose significant risks, including identity theft and unauthorized access to sensitive information.

# Functional Requirements: What It Does

### Password Generation

Generate random passwords with a mix of letters, numbers, and symbols.

### User Input Fields

Allow users to input website name, email, and generated password.

### Data Storage

Save website, email, and password details to a local text file.

### User Notifications

Provide feedback on save success or errors (e.g., empty fields).

# Non-Functional Requirements: How Well It Performs

### Usability

Intuitive and easy-to-navigate graphical interface.

### Security

Generated passwords must be cryptographically strong and unpredictable.

### Compatibility

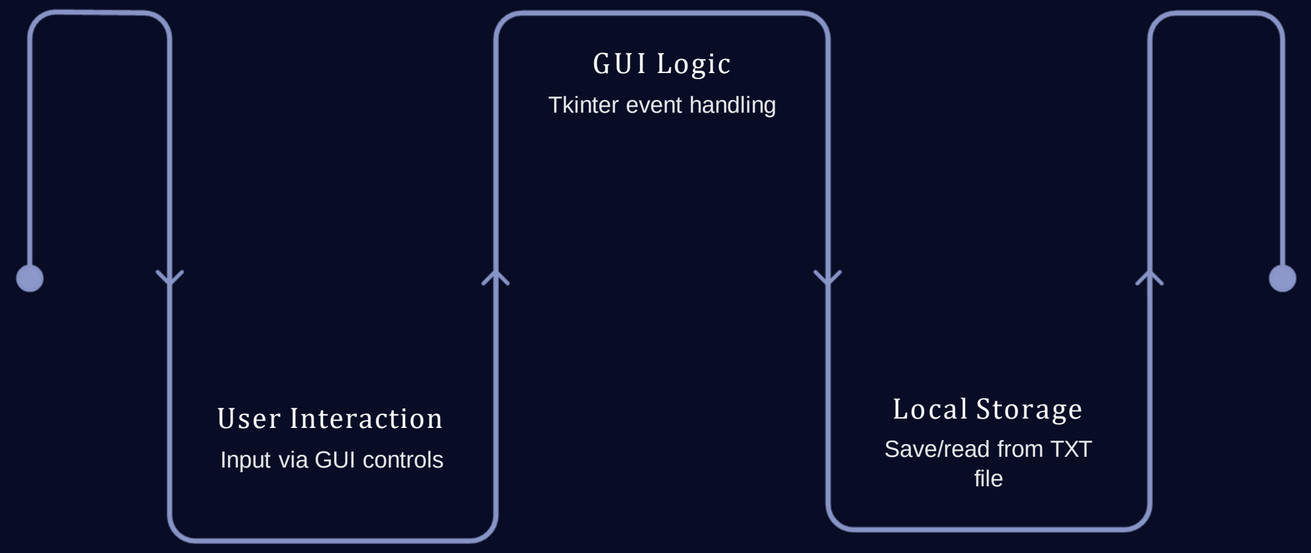
Compatible with standard desktop operating systems that support Python and Tkinter.

### Performance

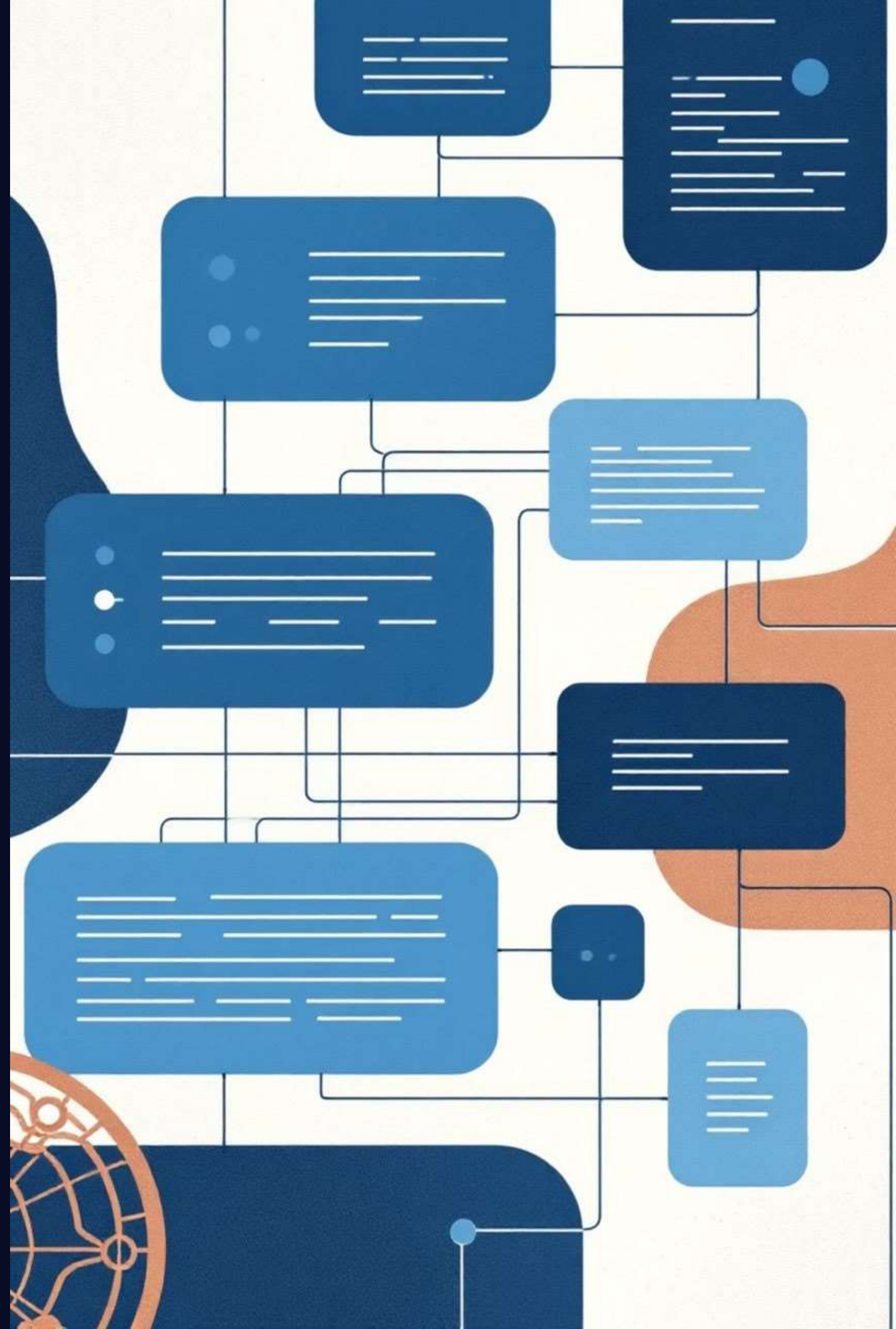
Rapid password generation and quick data saving operations.

# System Architecture

The system follows a simple client-side architecture where the Tkinter GUI handles user interaction and local file operations.




This design ensures a lightweight application with minimal dependencies.



# Design Diagrams: Visualizing the Flow

Understanding the application's flow and component interactions is crucial for effective development.

1

### Use Case Diagram

Illustrates user interactions: Generate Password, Save Credentials.

2

### Workflow Diagram

Maps the sequence of steps from input to saving data.

3

### Component Diagram

Shows the relationship between the GUI, generator, and file handler modules.

# Design Decisions & Rationale

### Tkinter for GUI

Chosen for its simplicity, cross-platform compatibility, and Python integration.

### Random Module for Passwords

Ensures unpredictability and strength of generated passwords.

### Text File Storage

A straightforward solution for local, non-sensitive data storage, ideal for personal use.

### Messagebox for Feedback

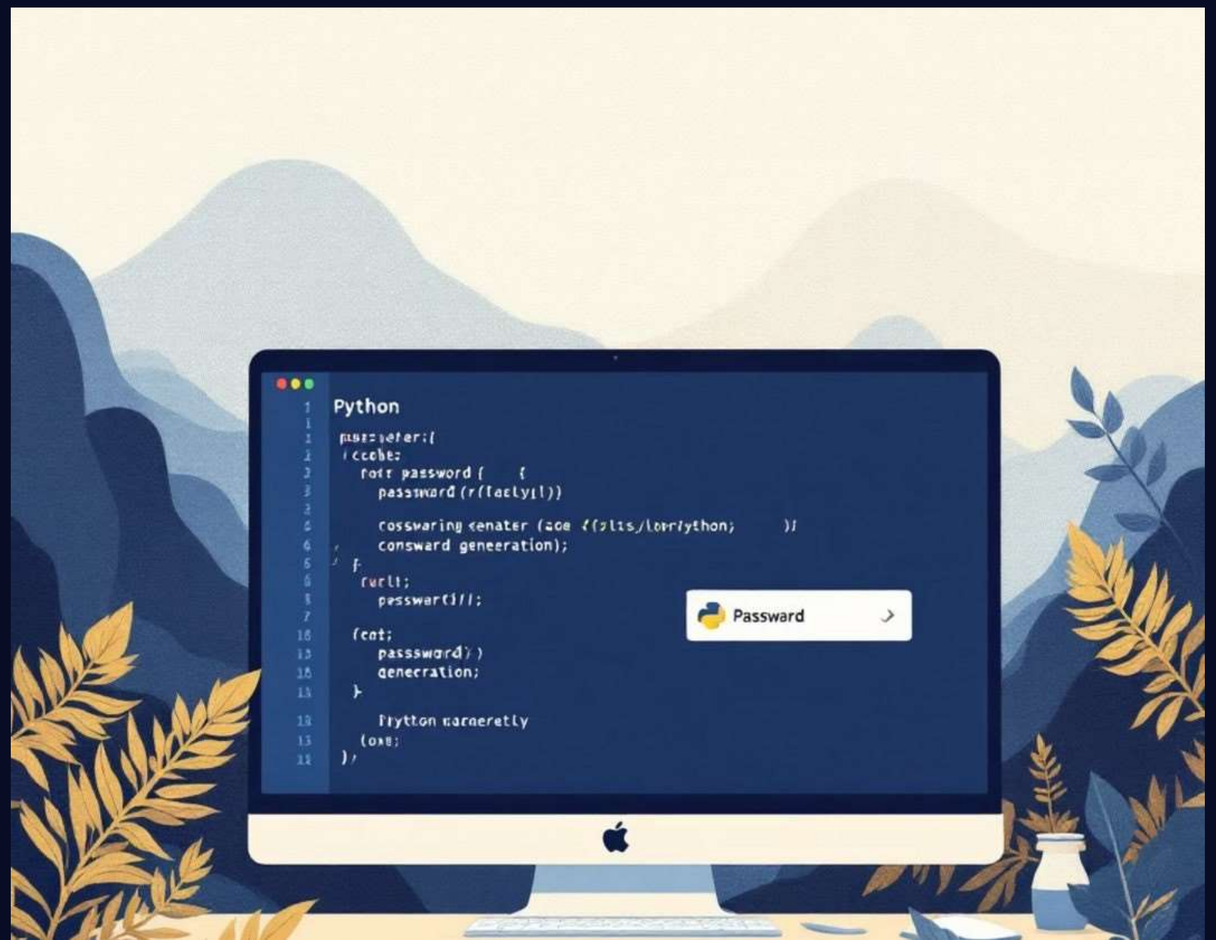
Provides immediate and clear communication to the user regarding actions and errors.

# Implementation Details: Key Code Snippets

The core logic revolves around password generation and data persistence.

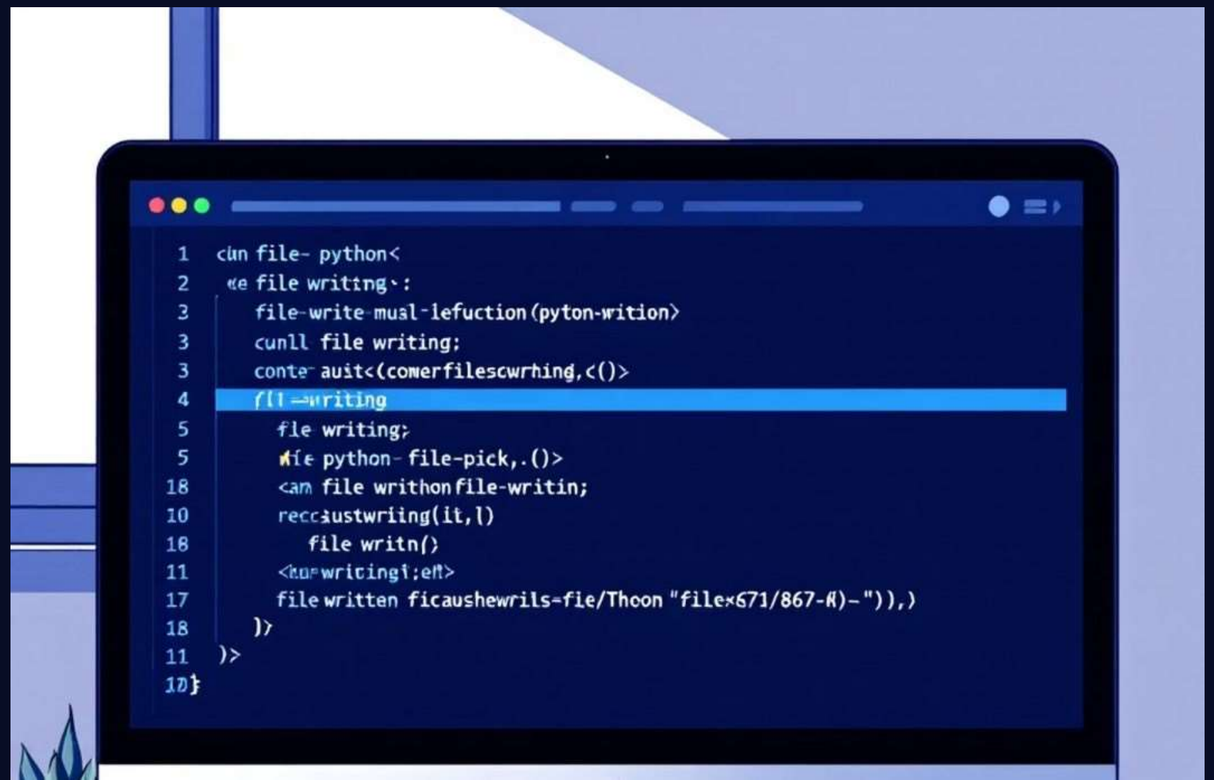
## Password Generation Function

```
def password_generator():    password_list = [...]    shuffle(password_list)    password =
"".join(password_list)    # Update UI
```



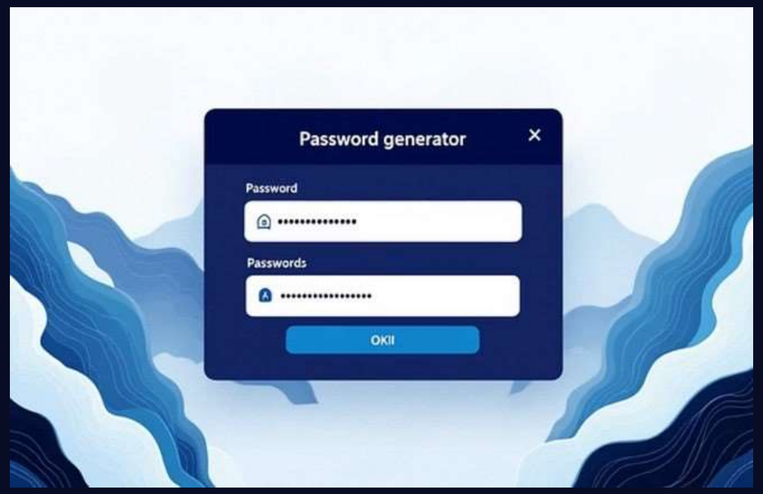

## Save Data Function

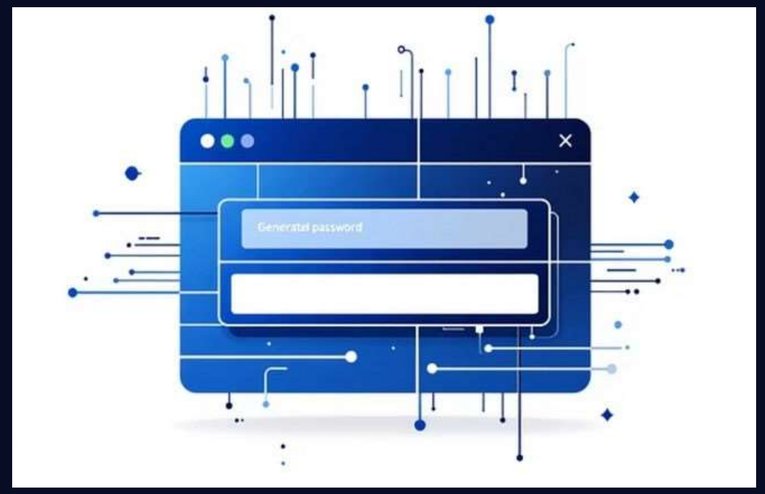
```
def save_data():    website = input_website.get()    email = input_email.get()    password =
input_password.get()    if ...:        messagebox.showinfo(...)        return    with open('password.txt',
'a') as data:        data.write(f"{website} || {email} || {password}\n")    # Clear fields
```



# User Interface: Screenshots & Results

The application provides a clean and functional interface for generating and saving passwords.






Users can quickly generate complex passwords and manage their credentials efficiently.